Regis University CPS
CS361 Programming Assignment 7
Due by midnight Tuesday of Week 8

Program #1

Terms in an **arithmetic series** are found by adding a constant value (known as the constant difference) to each prior term, starting with an initial value.

Example: Initial value = 1
Common difference = 3
Series: 1, 4, 7, 10, 13, 16, …

Note that the common difference is the difference between each term.
It can be positive or negative.

Write a program that when given a specific common difference will compute a specific term (i.e. the **n**th term) in an arithmetic series that starts with an initial term of 1.

You will implementing three functions that use different methods for computation.

Given:

$term_i$ = the **initial** term
$term_n$ = the **n**th term
d = the common difference

The first function (**CalcFormulaTerm**) will use the formula:

$$\boldsymbol{term_n = term_i + (n-1)d}$$

to compute and return the **n**th term.

The second function (**CalcLoopTerm**) will compute and return the **n**th term, using iteration to compute the term (i.e. a loop).

The third function (**CalcRecursiveTerm**) will compute and return the same **n**th term using recursion.

All three functions should take the ***common difference*** and ***term number to compute*** as input arguments and ***return*** the **n**th term in the arithmetic series with an initial term of 1.

The **main** function should first explain to the user what the program will do.

Then **main** should first read the **term number** to compute from the user, indicating that if the user enters 0, the program will exit. The code should error check the number entered is positive, and loop until the user enters a positive integer.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Unless the user enters 0 for the term number:

- The **main** function should then read the **common difference** to use in the series.

- After reading both inputs, the **CalcFormulaTerm**, **CalcLoopTerm** and **CalcRecursiveTerm** functions should all be called, passing the user inputs as input parameters to each.

- The results of all three function calls should be printed out by **main**, with descriptive text. Note that the results should be the same!

The program should loop and continue to compute results, ***until*** the user enters **0** for the value of the **term number**.

The **only** way the user may exit the program is to enter 0 for the value of the **term number** -- do not ask the user whether to run the program again.

Sample Run:

```
Program to compute a term in an arithmetic series starting with 1

Enter term number to compute (or 0 to exit):  5
Enter common difference to use: 5

Formula term =       21
Iterative term =     21
Recursive term =     21

Enter term number to compute (or 0 to exit):  10
Enter common difference to use: -2

Formula term =      -17
Iterative term =    -17
Recursive term =    -17

Enter term number to compute (or 0 to exit):  0
Goodbye!
```

Program #2

For program 2, you will modify program 2 from assignment 5, to incorporate re-use of code.

Program 2 from assignment 5 output accumulated values for each month, given a set amount saved each month, until the accumulated amount reached a set goal amount.

A banking firm purchased the code from another company. The new company would like you to rewrite the original program, to modify and enhance it.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Overview of Charges

- The new company offers information about loans, as well as savings.

- For loans, the program will determine the monthly loan payment for a given loan amount to be borrowed, and calculate total interest paid.

  The following formula will determine the loan payment:

  MonthlyPayment = (factor * monthlyInterestRate * loanAmount) / (factor - 1);

  Note that factor is computed the same way for a loan as it is for savings. The number of months in the factor calculation will be the number of months in the term of the loan.

- The savings display will be expanded to include a specific month and year, for each month displayed, and to include the total amount of interest earned.

Input Overview

The user will now get to choose whether to calculate when a savings goal will be reached, or when what a loan payment will be. A menu of choices will be displayed, including an Exit option. The program will repeat until the user chooses the Exit option from the menu.

The user must enter the numeric month and year that the first loan payment or savings deposit will be made. The user will also still input an amount (saved per month or loan total) and annual interest rate.

For savings, they will also enter the savings goal, and for loans, the user will also enter the length of the loan in years. But the upper and lower limits on each have changed:

Saved per month: 10 to 6000
Interest rate: 1% to 20%
Savings goal/loan amount: 100 to 900,000
Years loaned: 1 to 30

Sample Savings Input

```
Computation choices:
      S - Savings
      L - Loan
      E - Exit
Enter your choice: s

Enter 4-digit year savings will begin: 2014
Enter numeric month savings will begin: 3

Enter the annual interest rate: 2.7
Enter the dollar amount to be saved each month: 2
      Invalid amount -- must save between 10 and 6000 dollars a month.

Enter the dollar amount to be saved each month: 500
Enter the dollar amount of your savings goal: 2222
```

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Sample Loan Input

```
Computation choices:
      S - Savings
      L - Loan
      E - Exit
Enter your choice: l

Enter 4-digit year loan will begin: 2014
Enter numeric month loan will begin: 11

Enter the annual interest rate: 5.55
Enter the total loan amount to be borrowed: 9000
Enter number of years for loan: 4
```

Output Overview

The savings calculations will be the same as they were in assignment 5, program 2, The calculations will still cease displaying these accumulated savings when the total amount accumulated is *at or above* the goal.

The output will be similar to the output from assignment 5, program 2, except:

- Instead of just displaying a month number, the program will display a month name and a month year for each month until the goal amount has been accumulated.

- The summary following the monthly accumulations will be modified a bit, to show the total interest earned.

- The program will pause for each screen full of monthly accumulation totals.

  For a savings time of several years, the values may scroll by too quickly for the user to read them. Therefore your program will implement a pause after displaying each screen of monthly accumulation totals, as follows:

  a) Keep a count of the monthly accumulation lines displayed.
  b) Before displaying each line, check to see if the screen is full ("full" means approximately 20 lines have been displayed on the screen).
  c) Pause after each screen full of data has been displayed, so the user can read the information without it scrolling.
  d) After each pause, the column headers should be re-displayed, before displaying the next monthly accumulation line.

All dollar amounts will still be displayed to 2 decimal places, with the decimals lining up. The interest rate will still be displayed to 3 decimal places. The percent sign and “months” labels will still line up.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Sample Savings Output

```
                  Total
Month             Accumulated
------            -----------
2014 March             500.00
2014 April            1001.13
2014 May              1503.38
2014 June             2006.76
2014 July             2511.28

To reach your savings goal of      2222.00
With a monthly savings of           500.00
At an annual interest rate of         2.700 %
It will take                              5 months

Total savings deposits             2500.00
Interest earned                      11.28
                                  ----------
Total accumulated                  2511.28
```

The loan output will include a summary of the inputs, along with the computed monthly payments. It will also display the start and end dates of the loan, along with the total loan payments, and total interest paid. The data will be formatted the same way savings data is.

Sample Loan Output

```
For a loan amount of                  9000.00
At an annual interest rate of            5.550 %
And a loan period of                         4 years
Monthly payment will be                209.51

Loan will begin                 November 2014
Loan will end                    October 2018

Total loan payments                   10056.48
Original loan amount                   9000.00
                                  -----------
Total Interest paid will be            1056.48
```

Required User-defined Functions

1. A ***new*** function to read, validate, and return the menu choice.

2. A ***new generic*** function called **GetNum** that will prompt for, read, and validate a value. The function will have 3 parameters: a string prompt, a lower limit for valid values, and an upper limit for valid values. An error message should be displayed each time an invalid value is entered. Once a valid value has been entered, the function will ***return*** the validated value (a value between the lower limit and upper limit, inclusive).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

3. A ***revised*** function to read and validate the annual interest rate. Pass back ***both*** the yearly interest rate and the monthly interest rate, using reference parameters.

4. A ***new generic*** function called **GetStartDate** that will read and validate the month and year that the loan payments or savings will begin. The function will have one input parameter: a string description of whether the start date is for a loan or savings.

> The function will first prompt for, read, and validate a 4-digit start year, between the current year (defined in a constant) and 5 years in the future. An error message should be displayed each time an invalid year is entered, until a valid year is entered.
>
> After a valid year is entered, the function will prompt for, read, and validate a numeric start month (1 – 12), in a similar fashion.
>
> ***Both*** the start year and start month will be passed back, using **reference** parameters.
>
> NOTE: This function can call other functions.

5. The existing function to calculate and return the factor (from assignment 5)

6. A ***new*** function that is takes month number as an input parameter and returns the string name of the month for display.

7. A ***new*** function to determine the end date for a loan (month and year), given the start date (month and year) and number of years for loan repayment. The loan end month will be the month prior to the loan start month.

Example: Start month: May 2014
Length of Loan: 5 years
End month: April 2019

The end date values (month and year) should be passed back via ***reference*** parameters. Remember to account for the special case of loans that begin in January, and modify the month and year accordingly.

8. A new void function to process one loan request. This function will read all inputs, do the loan calculations, and display the loan results. Will call other functions to help do these tasks.

9. A new void function to process one savings request. This function will read all inputs, do the savings calculations, and display the loan results. Will call other functions to help do these tasks.

Program Execution

First display a program header and then display a menu to the user. Choices should include loan calculations, savings calculations, and exit the program.

Call the function to process the request.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Read the start date for the loan or savings, and the interest rate that will be used.

Then read the rest of the input needed for calculation requested.

For loans, call the generic function to read integers twice to:

Read the amount to be borrowed

Read the number of years for loan payback.

For savings, call the generic function to read integers twice to:

Read the amount to be saved

Read the total savings goal amount

After reading all of the input for one computation, perform the necessary computations and display the results.

After a pause, so the user can read the results , return to main and re-display the calculations menu.

Loop, until the user chooses **Exit** from the calculations menu.

Testing

As part of your submission for this week, you must write a test plan for this program. First list the rationale for creating your test cases. Then create each test case.

Your test plan must include both ***rationale*** for testing all conditions within the program and a list of the specific test cases. Each test case must include the exact ***user input used*** for the test and what the exact ***output expected***.

Submit your test plan as an MS Word document or an MS Excel spreadsheet.

Program and Function Documentation

1) Include **top of program** comments as specified in course Content section 1.8.

2) Include prototypes for ***all*** of your user-defined functions. The prototypes should be placed above **main**, and the function definitions should be placed below **main**.

3) Comment any constants/variables whose names are not completely descriptive.

4) Provide a comment above **each user-defined function definition** that includes:
   - What the function does
   - A description of the input parameters, used to pass values into the function
   - A description of the output parameters, used to pass values back from the function
   - A description of what is returned from the function via the return statement (if anything)

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Submission

This programming assignment is due by midnight Tuesday of Week 8.

Submit your program source code to the ***Prog Assn 7*** folder in the ***Dropbox*** area of the online course. **All** files should be attached to **one** submission for programming assignment 7.

Before submitting your files, you MUST name them as follows:

```
Lastname-wk8-prog1.cpp
Lastname-wk8-prog2.cpp
Lastname-wk8-p2testplan.xxx
```

Grading

The rubric that will be used to grade your program is included on the next pages

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

CS 361 Control Structures
Programming Assignment 7 Rubric

Program 1: (26 pts)				
Rating **Rating Category**	**Exemplary**	**Proficient**	**Basic (needs work)**	**Not Demonstrated**
Data Storage Points	Variables defined locally, using appropriate data types. Used descriptive names, following coding standards naming conventions.	A few minor errors in data typing, naming, or assignment.	One or more major errors, OR majority of names did not follow naming conventions.	Identifier names are not descriptive, and there is a multitude of errors OR defined global variable(s).
main function: User Input	Displays explicit prompts, and validates user input in a loop, re-prompting as necessary.	One minor error in the user input and validation code.	Multiple errors in the user input and validation code or input read, but not validated.	No input read from user - values possibly hardcoded.
Function Implementations: Formula	Function prototype and definition correct with proper parameter usage and a return value. Function logic produces correct results.	One or two minor error in function prototype or definition, or in function logic.	Multiple errors in function prototype or definition, or in function logic.	Required function was not implemented. Task was simply performed from within the main function or task not performed using formula at all.
Function Implementations: Iterative	Function prototype and definition correct with proper parameter usage and a return value. Function logic produces correct results. Loop statements are written correctly.	One or two minor errors in function prototype or definition, or in function logic or loop statements.	Many errors in function prototype or definition, or in function logic or loop statements.	Required function was not implemented. Task was simply performed from within the main function (1), or task not performed by looping at all (0).
Function Implementation: Recursive	Function prototype and definition correct with proper parameter usage and a return value. Function logic produces correct results. If statements and recursive call(s) are written correctly.	One or two minor errors in function prototype or definition, OR in function logic, recursive call, or if statements.	Many errors in function prototype or definition, OR in function logic, recursive call, or if statements.	Required recursive function was not implemented.
main function: Logic/Output	The logic accomplishes all program requirements and displays all of the required output. Loops correctly to repeat and exits only when 0 entered.	One or two minor errors in the main function logic loop, or output, or calls to other functions, and/or does not exit loop via test condition.	Many errors in the main function logic/output display, or function runs correctly ONCE, but does not loop.	Logic accomplishes none of the program requirements, and display completely incorrect output (or no output).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.

Program 2: (74 pts)				
Data Storage Points	Constants defined for ALL fixed values. Variables defined locally, using appropriate data types. Followed all coding standards naming conventions.	A few minor errors in data typing, naming, or assignment OR missing one or two necessary constants for fixed value.	One or more major errors, OR majority of names did not follow naming conventions, OR missing multiple constants for fixed values.	Constants not used, identifier names are not descriptive, and there is a multitude of errors OR defined one or more global variable(s).
Input Function: Menu	Function defined and called as specified by the requirements. Displays menu using constants for fixed values. Validates all user input. Passes back results.	One or two minor errors in function definition or call, or in the validation or error messages, or menu display uses hardcoded values instead of constant names.	Many errors in function definition or call, or in the user input and validation code or error messages.	Required function was not implemented. Task was simply performed from within the main function.
Input Function: Interest	Function prototyped, defined and called correctly. Displays explicit prompt and error messages. Passed back both annual and montly interest rates.	One or two minor errors in function prototype, definition, parameters, or calls.	Many errors in function prototype, definition, parameters, or calls.	Required function was not implemented. Tasks were simply performed from within the main function.
Input Function: Generic Integer	Generic function prototyped, defined and called correctly. Description and upper/lower limits passed as input arguments. Displays explicit prompt and error messages.	One or two minor errors in function prototype, definition (input parameters and return value), or calls, or in the user input and validation code.	Many errors in definition or calls, or defined multiple functions instead of one generic function, or implemented multiple functions within one using a big if-else statement.	Required function was not implemented. Tasks were simply performed from within the main function.
Input Function: Generic Date	Generic function prototyped, defined and called correctly. Input/output parameters correct. Displays explicit prompt and error messages. Called to read both savings and loan.	One or two minor errors in function prototype, definition, input/output parameters, or calls..	Many errors in definition or calls, or defined multiple functions instead of one generic function, or implemented multiple functions within one using a big if-else statement.	Required function was not implemented. Tasks were simply performed from within the main function.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Logic Function: Find Loan End Date	Prototyped, defined, and called correctly. All decision statements are written correctly and calculations produce correct results.	One or two minor errors in the prototype, definition, or calls.	Major or many errors in the prototype, definition, or calls.	Required function was not implemented, or tasks simply performed from within the main function.
Logic Function: Loan Request	Prototyped, defined, and called correctly. All logic is written correctly and produce correct results.	One or two minor errors in the prototype, definition, or calls.	Major or many errors in the prototype, definition, or calls.	Required function was not implemented, or tasks simply performed from within the main function.
Logic Function: Savings Request	Prototyped, defined, and called correctly. All logic is written correctly and produce correct results.	One or two minor errors in the prototype, definition, or calls.	Major or many errors in the prototype, definition, or calls.	Required function was not implemented, or tasks simply performed from within the main function.
Logic: main function	The program logic accomplishes all program requirements, calls other functions correctly, and loops correctly to repeat.	One or two minor errors in the main function logic loop or calls to other functions, or uses of break, return, or exit to exit a loop.	Many errors in the main function logic, OR function runs correctly ONCE, but does not loop.	Logic accomplishes few or none of the program requirements.
Output: Month number to word function.	Prototyped, defined, and called correctly. All if and/or switch statements are written correctly and produce correct results.	One or two minor errors in the prototype, definition, or calls.	Major or many errors in the prototype, definition, or calls.	Required function was not implemented, or tasks simply performed from within the main function.
Output: Display to User	Displays all of the required output, formatted according to program requirements.	One or two minor errors in the output display.	Many errors in the output display.	No output was displayed.
Program Modularity	Program logically broken down into at least main + 9 other logical functions. All functions defined and called correctly, with proper parameter passing.	One or two minor errors in the logical breakdown or in function definitions or calls, OR missing at least one required function.	Many errors in the logical breakdown or in function definitions or calls, OR missing multiple required functions.	There is no logic to the code included in each function, OR none of the required functions were implemented.
C++ Constructs / Readability / Miscellaneous	Code is organized and easy to follow. There is no unnecessary code and all code is spaced and indented correctly.	Code is fairly easy to read, but there are some spacing, indentation, and/or other minor issues, or extraneous code.	Code is somewhat difficult to read or has some other major issue.	The code is formatted so badly that it is extremely hard to read or has multiple major other issues.
Test Plan	Test plan includes rationale and tests that validate ALL logic.	Missing parts of rationale or missing tests of some logic.	Missing all rationale and/or missing many tests of the logic.	No test plan submitted.

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law

Both Programs				
Documentation	Top of program comments include all required components. Top of function comments explain what function will do, and describe all parameters and return values. Inline comments are included as necessary.	Program description missing some details about the input, output, or processing OR top of function comments vague or missing some details about parameters and return values OR missing other minor necessary comment.	Documentation has many problems with missing, incomplete and/or incorrect items in program description, top of function comments, and/or inline comments.	Only a few (or no) comments in program.

NOTE: Deductions for late submissions will be applied (-2% per day late)
But programs will **not be accepted** if they are submitted **more than 5 days** late (grade will be 0).

©2014, Regis University, All Rights Reserved
Unauthorized distribution (including uploading to any non-Regis Internet website) violates copyright law